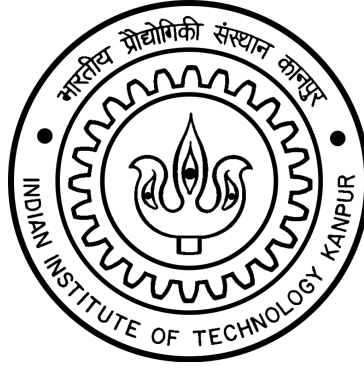




CS697: SPECIAL TOPICS IN COMPUTER SCIENCE

Serverless Execution of Time-Bound C Programs with Multi-Node Support

Team Members:

Sparsh Sehgal (241110071)
Vinayak Devvrat (241110079)

April 30, 2025

Contents

1	Introduction	3
2	Installation	4
2.1	Pre-requisites	4
2.2	Setup Details	4
2.2.1	CouchDB Installation and Setup	5
2.2.2	OpenWhisk Installation	8
3	OpenWhisk Components	14
3.1	OpenWhisk Load Balancer Policies	14
3.1.1	FPCPoolBalancer Introduction	14
3.1.2	Data Structures and Variables Used	14
3.1.3	Overview of OpenWhisk Schedulers	15
3.1.4	Overall Workflow Diagram	16
3.1.5	Step-by-Step Workflow Details	17
3.1.6	Invoker selection and Container selection	19
3.2	What Consititutes a “Healthy” Invoker ?	19
4	Profiling with Existing Workloads	22
4.1	Configuration	22
4.2	Workload Suites	22
4.3	Results	23
4.3.1	Matrix Multiplication (Matmul)	23
4.3.2	Solving Linear Equations (Linpack)	25
4.3.3	Inferences	26
5	Implementation	29
5.1	Creating Docker Container for C Support	29
5.1.1	Approach 1	29
5.1.2	Approach 2	30
5.1.3	Approach 3	30
5.2	Implementing the Activation Code(s)	30

5.3	Imposing Time Limits on Executions	30
5.3.1	Timing out the entire Container	31
5.3.2	Timing out a Specific Test Case	31
5.3.3	Customising Time Limits for each Test Case	32
6	Results	33
6.1	Testing Timeout Scenarios	33
6.2	Profiling the Implementation	36
6.2.1	Evaluating the Timing	36
6.2.2	Evaluating the CPU Usage	36
7	Conclusions and Future Work	39

Chapter 1 Introduction

This project focuses on the deployment, customisation, and performance evaluation of Apache OpenWhisk, an open-source serverless computing platform, on a multi-node Ubuntu server environment. The internal workflow and architecture of OpenWhisk are examined in detail, with focus on its principal components such as the scheduler, controller, and invoker. The load balancer policy of OpenWhisk is also studied.

After successful installation, a series of benchmarking experiments are conducted using computationally intensive tasks such as matrix multiplication and the solution of linear systems of equations. These experiments are designed to evaluate the platform's response time and CPU utilization across various conditions.

To extend the functionality of the system, modifications are made to the existing Python-based Docker containers to incorporate support for C/C++ applications through the integration of the GNU Compiler Collection (GCC). With this customised environment, various C programs are executed, and multiple techniques are implemented to impose execution time limits within the containers. The performance of these executions is again profiled and analysed including response time and CPU usage. This comprehensive evaluation provides insights into the adaptability, efficiency, and resource management capabilities of OpenWhisk when supporting workloads beyond traditional serverless paradigms.

Chapter 2 Installation

For this project, we have setup OpenWhisk on Ubuntu 22.04. The pre-requisites are essential for a multi-node OpenWhisk setup on Ubuntu server(s).

2.1 Pre-requisites

Each node should have support for Docker, Java 8, NodeJS and Python3.

Memory Requirements

- Each node requires at least 4 GB of memory and 2 CPU cores.
- The storage for the primary node should be at least 25 GB and 40 GB for each secondary node.

Other Requirements

- `sudo` privilege on all three nodes is required.
- The primary node user should be able to ssh to the secondary nodes without password.
- The user for each node should be part of the `docker` group.

2.2 Setup Details

To conduct our experiments, we used the following setup:

- **Total number of nodes : 3**
 - 1 primary node for hosting the scheduler, controller, nginx and other dependent containers.
 - 2 secondary nodes with one invoker container each. Each secondary node will also host containers for Python, NodeJS, Java etc when action pertaining to any of these containers are invoked.

- CouchDB in **clustered mode** is installed on each node with the primary node serving as the **Admin** node.

2.2.1 CouchDB Installation and Setup

The steps in this section are to be executed in all three nodes unless stated otherwise.

2.2.1.1 Installing CouchDB

Step 1

In a clustered setup, the nodes need to communicate with one another. CouchDB achieves this communication using Erlang. Install Erlang using:

```
sudo apt-get install erlang
```

To verify communication between the nodes via Erlang refer to steps given in the Appendix.

Step 2

Run the following command to install the required tools for the CouchDB installation:

```
sudo apt update && sudo apt install -y curl apt-transport-https gnupg
```

Step 3

Now, add CouchDB's GPG key and convert it into a format apt can use:

```
curl https://couchdb.apache.org/repo/keys.asc | gpg --dearmor | sudo tee /usr/share/keyrings/couchdb-archive-keyring.gpg >/dev/null 2>&1
```

Step 4

Based on the OS information (such as `$VERSION_CODENAME`), relevant CouchDB packages must be downloaded. This is done using the following commands:

```
source /etc/os-release
```

```
echo "deb [signed-by=/usr/share/keyrings/couchdb-archive-keyring.gpg] https://apache.jfrog.io/artifactory/couchdb-deb/ $VERSION_CODENAME main" | sudo tee /etc/apt/sources.list.d/couchdb.list >/dev/null
```

Step 5

CouchDB is now ready to be installed. We use:

```
sudo apt install -y couchdb
```

At the end of the installation, we need to select the type of setup: **Standalone**, **Clustered** or **None**. Since, we need to customise our setup for OpenWhisk, we select **None**.

2.2.1.2 Cluster Setup

This section details the steps to setup the three nodes in a cluster. Execute the steps with `sudo`.

Step 1

Navigate to `/opt/couchdb/etc` directory. Make the following changes in `vm.args`:

```
1 -name couchdb@<node-ip>
2 -setcookie '<cookie-name>' #This has to be same for all three
   nodes
3 ...
4 -kernel inet_dist_use_interface {0,0,0,0}
5 -kernel inet_dist_listen_min 9100
6 -kernel inet_dist_listen_max 9200
```

Additionally, make the following changes `local.ini`:

```
1 [chttpd]
2 ;port = 5984
3 bind_address = 0.0.0.0
4 ...
5 [admins]
6 admin = <admin-password> ; Set a password for admin user
```

Step 2

Navigate to `/etc/default` and make the following change in `couchdb`:

`ERL_EPMD_ADDRESS=<node-ip>`

`ERL_EPMD_ADDRESS` specifies the IP address that CouchDB (specifically, its Erlang runtime) should use to bind Erlang Port Mapper Daemon (epmd).

Step 3

In CouchDB, the `_users` database is a special system database used for storing user accounts such as the admin. It is created by executing the following:

```
curl -X PUT http://admin:<admin-password>@<node-ip>:5984/_users
```

At this point, it is important to make the following verifications:

- Verify the status of the `couchdb` service using
`sudo systemctl status couchdb`
The status should be **active (running)**.

- Navigate to `/opt/couchdb/var/log` directory, open the file `couchdb.log`. Verify the messages that indicate a successful setup.

Step 4

This step must be executed only on the primary node.

For the primary node to also be the Admin node, it must generate two Universally Unique Identifiers (UUIDs). The same UUIDs will be used on all the nodes. Generate the two UUIDs using:

```
curl http://<server-IP|FQDN>:5984/_uuids?count=2
```

Copy the provided UUIDs into a text editor for later use.

Step 5

This step must be executed on all the nodes.

Set the `uuid` and `secret` using the following:

```
1 curl -X PUT
   http://<node-ip>:5984/_node/_local/_config/couchdb/uuid -d
   ' "UUID1" '
2 curl -X PUT
   http://<node-ip>:5984/_node/_local/_config/httpsd_auth/secret
   -d ' "UUID2" '
```

Step 6

This step must be executed only on the primary node.

CouchDB provides a web interface called Fauxton which can help in performing CRUD operations on databases. It is hosted at `http://<node-ip>:5984/_utils`.

1. Open the url above and click on “Setup“ tab in the navigation panel.
2. Click on “Setup Cluster” and enter the admin credentials again.
3. Enter each node’s IP address in the setup wizard along with their ports and the previously set server admin username and password.
4. Once all the nodes have been added, click “Setup“ and Fauxton will finish the cluster configuration.
5. In the terminal, verify the cluster configuration using the following on any node:
`http://<server-IP|FQDN>:5984/_membership`.
The JSON returned from the above call should have the IP addresses of all the nodes in the `"all-nodes"` and `"cluster-nodes"` keys.

Step 7

In this step, we verify if the cluster has been set up successfully.

1. Choose any arbitrary node from the cluster.
2. Create a database (say `temp`) on this node using either a PUT call similar to the one used for `_users` or using Fauxton.
3. Verify that `temp` has also been created on all nodes.

CouchDB has now been successfully setup in clustered mode.

2.2.2 OpenWhisk Installation

2.2.2.1 Setup the Server

After having successfully installed CouchDB based on the instructions in the previous section, we now follow the steps described in the upcoming sections to install OpenWhisk.

Please note that the steps described below must be followed only for the Primary node unless stated otherwise.

Step 1

Clone the OpenWhisk repository:

```
git clone https://github.com/apache/openwhisk.git
```

Step 2

As indicated earlier, OpenWhisk now requires Python3. However, the repository uses some library versions and commands that are no longer compatible with Python 3. Make the following changes in the files specified to avoid encountering errors in subsequent steps.

1. In the file `tools/ubuntu-setup/ansible.sh`, replace
`sudo apt-get install -y python-dev libffi-dev libssl-dev`
with
`sudo apt-get install -y python3-dev libffi-dev libssl-dev`
Additionally, make the following version changes:

```
1 sudo pip install ansible==4.1.0
2 sudo pip install jinja2==3.0.1
3 sudo pip install docker      --ignore-installed
   --force-reinstall
```

```
4 sudo pip install httplib2 --ignore-installed
   --force-reinstall
5 sudo pip install requests==2.29.0 --ignore-installed
   --force-reinstall
```

2. In the file `tools/ubuntu-setup/pip.sh`, replace the line:

```
sudo apt-get install -y python-pip
with
sudo apt-get install -y python3-pip
```

Step 3

We are customising Scheduler(s) for our setup. To that end, we need make the following replacement in the `whisks spi` object to have a compatible implementation of `DurationCheckerProvider`. In the file, `common/scala/src/main/resources/reference.conf`, make the following change:

Replace
`DurationCheckerProvider =`
`org.apache.openwhisk.core.scheduler.queue.ElasticSearchDurationCheckerProvider`
in `whisks spi` with
`DurationCheckerProvider =`
`org.apache.openwhisk.core.scheduler.queue.NoopDurationCheckerProvider.`
At the end of this step, our setup is ready to import and install the necessary dependencies.

Step 4

As the root user execute the following command:

```
(cd tools/ubuntu-setup && ./all.sh)
```

Step 5

OpenWhisk relies heavily on Ansible which is a suite of software tools for configuring and deploying applications. To establish a connection between the CouchDB and OpenWhisk, Ansible requires database credentials. These credentials can be specified as environment variables which can then later be picked up by Ansible when we execute the relevant `.yaml` file. Initialise the following variables:

```
1 export ENVIRONMENT=local
2 export OW_DB=CouchDB
3 export OW_DB_USERNAME=<admin-username>
4 export OW_DB_PASSWORD=<admin-password>
```

```
5 export OW_DB_PROTOCOL=http
6 export OW_DB_HOST=<node-ip>
7 export OW_DB_PORT=5984
```

Step 6

OpenWhisk supports multiple schedulers, controllers and invokers that can be hosted on multiple nodes. We need to specify the host ip for each of these in the `hosts.j2.ini` file. This file can be found in the `ansible/environments/local/` directory. Make the following changes in `hosts.j2.ini` file:

1. Replace all occurrences of the IP address `172.17.0.1` with the IP address of the primary node.
2. In our experiments, we use two invoker nodes hosted on two secondary nodes. Therefore, the following specifications need to be made under `[invokers]`:

```
1 [invokers]
2 invoker0 ansible_host=<secondary-node1-ip>
   ansible_connection=ssh
3 invoker1 ansible_host=<secondary-node2-ip>
   ansible_connection=ssh
```

Similar changes would be made if scheduler(s) and/or controller(s) are hosted on different nodes.

Step 7

Change the current working directory to `ansible` by executing
`cd ansible`

Now, execute the following commands:

1. `ansible-playbook setup.yml`
2. `ansible-playbook prereq.yml`

At the end of the first command, a file called `db_local.ini` should get created. To verify the success of this command, it can be checked that the credentials specified as environment variables are same as the credentials mentioned in the file.

Step 8

After successful runs of both these commands, traverse one directory back i.e. `openwhisk` and execute the following command:

```
./gradlew distDocker
```

This will take some time as the required docker images for controller, scheduler, invokers and other components are downloaded besides some other dependencies such as Scala.

Step 9

In this step, we create a docker registry to store the images for controller, invoker and scheduler. This is especially useful in cases of version changes at OpenWhisk's end and also gives the developers a tighter control over the dependencies.

1. Create the docker registry that is hosted at a given port (5000 in our case) using
`docker run -d -p 5000:5000 --name registry registry:2`
2. Tag the scheduler image and push it to the repository using
`docker tag whisk/scheduler <primary-node-ip>:5000/whisk/scheduler`
`docker push <primary-node-ip>:5000/whisk/scheduler`
This has to be repeated for invoker and scheduler images as well.

Edit the `docker-registry` variable in `ansible/environments/local/group_vars/all`:
`docker-registry: <primary-node-ip>:5000/`

Step 10

In this step we execute the `.yaml` files responsible for creating and initialising the databases that OpenWhisk will access from time to time for various purposes such as storing and getting activations. Run the following commands:

1. `ansible-playbook initdb.yaml`
2. `ansible-playbook wipe.yaml`

The first command, initialises the relevant databases such as `whisk_local_activations` if they have not been initialised before.

The second command removes any existing entries present in the databases if they were already existed.

Since the second command removes any existing data, it must not be executed again unless the existing data is no longer required.

Step 11

This is the most crucial step of the installation. Upon successfully executing this `yml`, all the relevant containers should be up and running on their respective nodes. The invoker nodes would host containers for Python, Swyft, Java among others. This step also will take some time.

Since we are using CouchDB for storing the activations, the `activation-store` must be specified in scheduler configuration. Hence, specify the following in `whisk` in the file `openwhisk/core/scheduler/src/main/resources/application.conf`:

```
1 activation-store {  
2     couchdb {  
3         protocol = "http"  
4         host = "<primary-node-ip>" # CouchDB host  
5         port = 5984 # CouchDB port  
6         dbname = "whisk_local_activations" # Database name  
7         username = "<admin-username>" # CouchDB username  
8         password = "<admin-password>" # CouchDB password  
9     }  
10 }
```

Now, execute the following:

```
ansible-playbook openwhisk.yml -u <secondary-node-username> --key-file  
<path/to/id_rsa>
```

For simplicity, in our experiments we have kept the username of both secondary nodes same. If both nodes have different usernames, however, they must be specified using the variable `ansible_user` in the `hosts.j2.ini` file along side `ansible_host` in Step 6.

After this step, all the containers in all three nodes should be up and running. This can be verified using `docker ps -a` command.

Step 12

Execute the following commands:

1. `ansible-playbook postdeploy.yml -u <secondary-node-username>
--key-file <path/to/id_rsa>`
2. `ansible-playbook apigateway.yml -u <secondary-node-username>
--key-file <path/to/id_rsa>`
3. `ansible-playbook routemgmt.yml -u <secondary-node-username>
--key-file <path/to/id_rsa>`

The first command installs some more useful packages for OpenWhisk such as the `wsk` executable in the `openwhisk/bin` directory to execute OpenWhisk CLI commands.

As mentioned in Section 2.2, the three nodes will host OpenWhisk while the client will be

a different machine. For client-server communication, the API Gateway must be enabled. This is achieved by commands 2 and 3.

After following all the steps mentioned in this section a multi-cluster OpenWhisk setup should be up and running.

2.2.2.2 Setup the Client(s)

We follow the following steps to install OpenWhisk CLI on the client machine.

1. Download the CLI zip from [here](#).

2. Extract the `wsk` binary from the zip and move it to `/usr/local/bin`.

3. Verify the installation using the following command:

```
wsk -v
```

4. Set the api host and authorisation key for OpenWhisk using the following commands:

```
wsk property set --apihost <primary-node-ip>
```

```
wsk property set --auth 'cat
```

```
/path/to/openwhisk/ansible/files/auth.whisk.system'
```

The path in the command above refers to the path of the `openwhisk` directory in the primary node.

5. Test the success of the setup by executing the following:

```
wsk action invoke /whisk.system/utils/echo -p message hello --result -i
```

It should return:

```
1 {  
2   "message": "hello"  
3 }
```

OpenWhisk is now setup for use.

Chapter 3 OpenWhisk Components

3.1 OpenWhisk Load Balancer Policies

OpenWhisk has many choices for the Load Balancer such as FPCPoolBalancer, ShardingContainerPoolBalancer, etc. Default Load Balancer is FPCPoolBalancer.

Which pool balancer to employ can be configured by updating *LoadBalancerProvider* property in file *openwhisk/common/scala/src/main/resources/reference.conf*

3.1.1 FPCPoolBalancer Introduction

FPCPoolBalancer is responsible for publishing activation messages to a Kafka topic corresponding to the appropriate scheduler endpoint. It also sets up activation state, handles timeouts, and processes acknowledgements from invokers.

Source file - `FPCPoolBalancer.scala`

3.1.2 Data Structures and Variables Used

- **schedulerEndpoints**: A `TrieMap[SchedulerEndpoints, SchedulerStates]` mapping. This holds the endpoints of available schedulers (which are separate components managing activation queues) along with their state (including metrics such as queue size).
- **queueEndpoints**: A `TrieMap[String, SchedulerEndpoints]` used to map a queue key (derived from invocation namespace and action name) to the responsible scheduler endpoint. Example of a queueKey -

```
Namespace: guest
Action: helloWorld
queueKey = queue/guest/helloWorld/leader
```

(If the action lives in a package, the fully qualified name is used, e.g. `queue/guest/pkgName/actionName/leader`.)

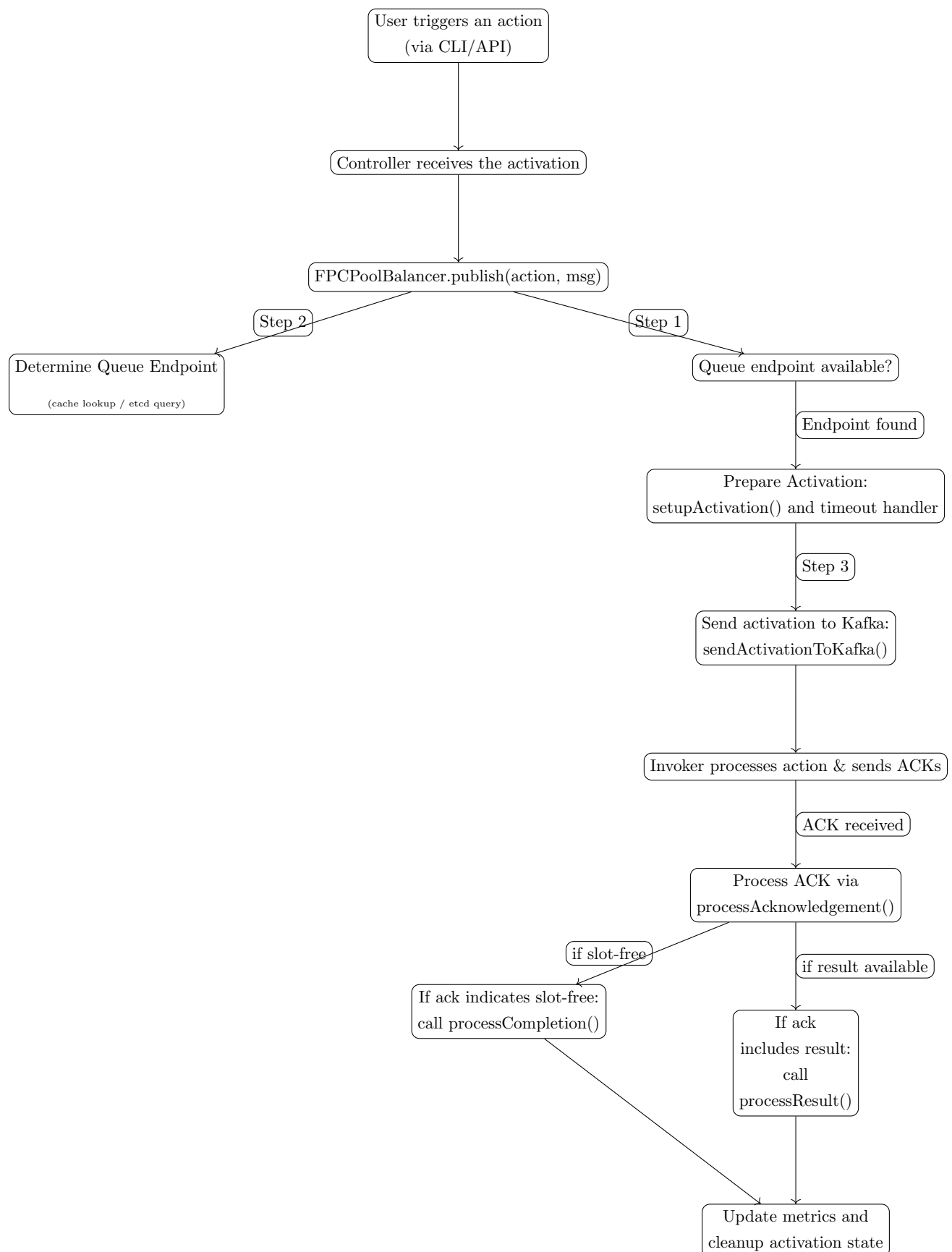
- **activationSlots:** A `TrieMap[ActivationId, DistributedActivationEntry]` that stores the state of in-flight activations. Each entry holds information like memory limit, concurrency limit, timeout handler, and associated namespace.
- **activationPromises:** A `TrieMap[ActivationId, Promise[Either[ActivationId, WhiskActivation]]]` used for blocking invocations. When an action is invoked in blocking mode, a promise is stored so that its result can be returned later.
- **Counters:** Variables such as `activationsPerNamespace`, `activationsPerController`, `totalActivations` and `totalActivationMemory` are used to track ongoing invocations and resource (memory) usage.

3.1.3 Overview of OpenWhisk Schedulers

OpenWhisk is built on a distributed architecture where scheduling plays a key role in routing activations to available invokers. Some key points:

- **Role:** Schedulers are separate components (often deployed as dedicated processes or containers) that manage activation queues. They receive activations from the load balancer and assign them to invokers.
- **Management:** Scheduler instances register their state (encapsulated in `SchedulerStates`) in a distributed key-value store (etcd). This information is maintained in the `schedulerEndpoints` mapping within the load balancer.
- **Number and Naming:** The number of active schedulers in a deployment is configurable and depends on the cluster design (typical deployments might have one per controller or more, depending on load). Each scheduler is identified by a unique `SchedulerInstanceId` and its endpoint (e.g., "scheduler0", "scheduler1").
- **How They Work:** When a scheduler starts, it registers its endpoint and state. The `FPCPoolBalancer` uses these endpoints to determine where to route a specific activation. It may choose the scheduler with the smallest queue size via a scheduling algorithm.
Each scheduler endpoint (e.g. scheduler0, scheduler1, etc.) runs its own `QueueManager` which holds one queue per (namespace + action) pair. Multiple activations for the same action are enqueued in that scheduler's local queue. Schedulers maintain real-time metrics such as queue size.
- **Coordination:** A dedicated watcher actor in `FPCPoolBalancer` listens to etcd changes (on keys with prefixes like `SchedulerKeys`) and updates the `schedulerEndpoints` mapping dynamically.

3.1.4 Overall Workflow Diagram



3.1.5 Step-by-Step Workflow Details

Step 1: Activation Message Reception and Queue Endpoint Determination

When the controller invokes the pool balancer, the `publish` method is called.

Branch:

- **If no scheduler endpoints are available:** The method fails and logs an error.
- **If the queue endpoint is already cached:** It immediately uses the cached endpoint.
- **If not cached:** It queries etcd for the key and updates the cache.

Step 2: Setup Activation State and Timeout Handler

Before sending the message to Kafka, the balancer sets up the activation state.

```
1 val resultPromise = if (msg.blocking) {
2   activationPromises.getOrElseUpdate(msg.activationId,
3     Promise[Either[ActivationId, WhiskActivation]]()).future
4 } else Future.successful(Left(msg.activationId))
5
6 activationSlots.getOrElseUpdate(
7   msg.activationId, {
8     val timeout =
9       (action.limits.timeout.duration.max(TimeLimit.STD_DURATION)
10        * lbConfig.timeoutFactor) + 1.minute
11     val timeoutHandler =
12       actorSystem.scheduler.scheduleOnce(timeout) {
13         processCompletion(msg.activationId, msg.transid, forced =
14           true, isSystemError = false)
15       }
16     DistributedActivationEntry(
17       msg.activationId,
18       msg.user.namespace.uuid,
19       msg.user.namespace.name.asString,
20       msg.revision,
21       msg.transid,
```

```
22     msg.blocking,
23     controllerInstance)
24 }
25 )
```

Explanation:

- Increments counters (stored in `totalActivations` and `totalActivationMemory`).
- If the activation is blocking, a promise is created and stored in `activationPromises`.
- Schedules a timeout handler to invoke `processCompletion` with `forced=true` if no ACK arrives in time.
- Creates a `DistributedActivationEntry` in `activationSlots` to store all activation-related metadata.

Step 3: Sending the Activation to Kafka

After setting up the activation, the message is sent to Kafka.

Decision Branch:

- **Success:** The Kafka message is published and the event is logged.
- **Failure:** An error is logged via the transaction logging system.

Step 4: Processing Acknowledgements from Invokers

An activation feed subscribes (subscribed to Kafka) to acknowledgements sent from invokers.

Branch:

- **If parsing succeeds:**
 - If `isSlotFree` is present, `processCompletion` is invoked.
 - If a result is provided, `processResult` completes the promise.
 - If a result is present, `processResult` is called to complete the promise for blocking invocations.
- **If parsing fails:** An error is logged and marks the message as processed.

Step 5: Handling Completion via processCompletion()

When a completion ack (or a forced timeout) occurs, the activation state is cleaned up.

Branch:

- **Activation Entry Found:**

- If the ack is *normal* (not forced), the timeout is canceled, decrements counters, and metrics are updated.
- If the ack is *forced* (due to timeout), an error is logged, emits the appropriate metric, and fails the promise.

- **No activation state present:**

- When the ack arrives after the entry is already removed (e.g., late acknowledgement), it logs a warning or debug message and emits a different metric.

3.1.6 Invoker selection and Container selection

Scheduler sees which invoker is healthy and have free slots, picks one, and launches—or reuses—a container there.

On each selected invoker, activations are routed into a `ContainerPool` actor which in turn chooses—or creates—a container.

- The pool first tries to find an existing warm container whose invocationNamespace and action match and which still has capacity.
- If no in-use container fits, the pool picks the oldest pre-warmed container matching the action's kind.
- If still no match then new container is cold started.

3.2 What Constitutes a “Healthy” Invoker ?

An invoker's state can be categorised into “Healthy”, “Unhealthy” and “Offline”. The state of a newly created invoker is “Unhealthy”. A **health test** is performed by the `InvokerHealthManager`. It makes use of a boolean array called the `RingBuffer` that holds the results of last 10 health tests. The health test itself has the following steps:

- The `InvokerHealthManager` fetches the default Node.js runtime from OpenWhisk's runtime manifest.
- It then executes a lightweight action on the newly created container. This is referred to as the test action.

- The result of the invocation of the above action is stored in `RingBuffer`.
- If the number of false counts in the `RingBuffer` is greater than 3 then the invoker is marked as “Unhealthy”.

Memory limits are placed upon the container so that it does not consume significant CPU or memory due to this testing.

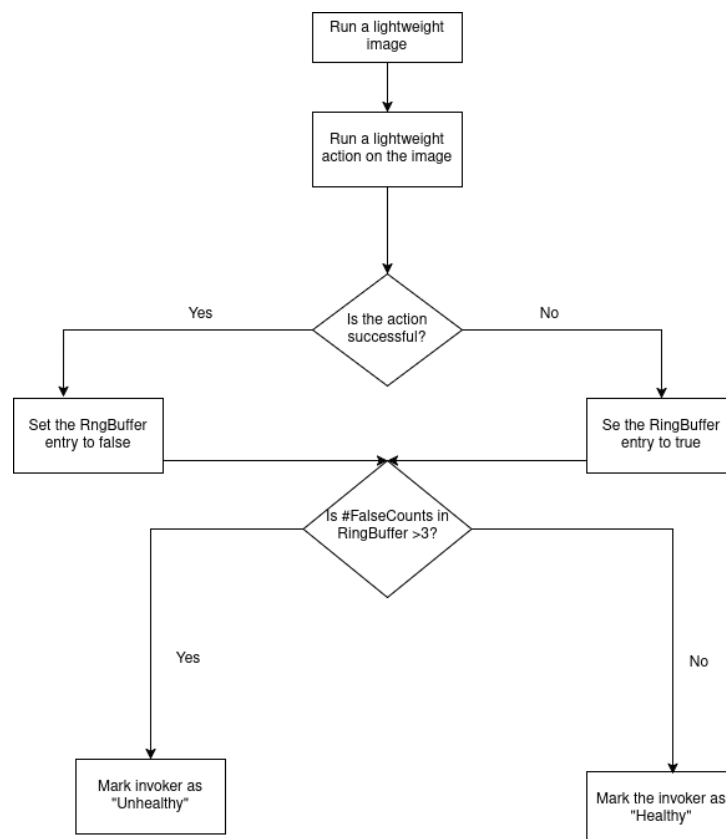


Figure 3.1: Invoker Health Test

The `InvokerHealthManager` also monitors the following state changes of the invoker:

1. **Offline to Unhealthy.** This transition takes place when an invoker is enabled by the triggering of an action.
2. **Unhealthy to Offline.** This transition takes place when a container on which a test action is to be executed has been removed. The invoker may execute the test action again to detect recovery.
3. **Healthy to Unhealthy.** This state transition happens when the number of health check failures in place have crossed a threshold.

If any of the above conditions hold true, `InvokerHealthManager` runs the health test again.

How does the Scheduler delegate Activations to Invoker(s) ?

The scheduler first filters all the invokers based on the following three parameters:

1. **Invoker State.** The invoker should be in “Healthy” state.
2. **Minimum Memory Requirement.** The size of the invoker’s free memory should be greater than or equal to the memory required by the activation.
3. **Namespace affinity.** The invoker should not be dedicated to a particular namespace (general purpose) or should be dedicated to the invocation namespace.

From the list filtered invokers, the scheduler further checks for each invoker, if a given activation request can be scheduled on the invoker’s **warmed creations** i.e. the containers that are already initialised with the action code or if it would have to create new container i.e. **cold creations**.

The memory usage of the selected invoker is subsequently updated in the invoker’s meta-data.

Chapter 4 Profiling with Existing Workloads

4.1 Configuration

We are essentially stress testing our OpenWhisk setup. The objective is to maximise the number of parallel invocations of the workloads to assess the load balancing and scalability capabilities of OpenWhisk.

To establish the benchmarks for reasons stated previously, we change the default limits set by OpenWhisk on number of invocations per minute, allowed number of concurrent invocations and so on. Navigate to `ansible/group_vars/`. Make the following changes in the `all` file:

```
1 limits:
2   invocationsPerMinute: "{{ limit_invocations_per_minute |
3     default(100000) }}" #default was 60
4   concurrentInvocations: "{{ limit_invocations_concurrent |
5     default(50000) }}" #default was 30
6   firesPerMinute: "{{ limit_fires_per_minute | default(100000)
7     }}" #default was 60
8   sequenceMaxLength: "{{ limit_sequence_max_length |
9     default(5000) }}" #default was 50
```

Restart `openwhisk.yml` and wait for all the containers to come up.

4.2 Workload Suites

We make use of the workload suites provided in [2] and have also been used in [1]. The workloads we use specifically benchmark the latency, CPU and memory bound performance. The details of the workloads are as follows:

1. **Matrix Multiplication.** Multiplication of two $N \times N$ square matrices. For the purposes of our experiment, $N = 200$.
2. **Solving Linear Equations.** Solving a set of N linear equations of the form $Ax = B$. For the purposes of our experiments, $N = 200$.

4.3 Results

For profiling, we first cloned the GitHub repository mentioned in [1]. Using the following commands, we created activations with the customised Docker images provided in the repository:

```
1 wsk action create matmul --docker
  andersonandreii/python3action:matmul cpu-memory
  matmul/function.py -i
2 wsk action create matmul --docker
  andersonandreii/python3action:matmul
  cpu-memory/linpack/function.py -i
```

These commands created two activations with an action code that was specified in the respective `function.py` files.

To profile the setup more robustly, we invoked the actions `matmul` and `linpack` concurrently with a bash script. The number of concurrency invocations were started at 10 then increased to nearly 900 or till the setup crashed.

The figure 4.1, gives a comparative analysis of the benchmarks over several concurrent invocations.

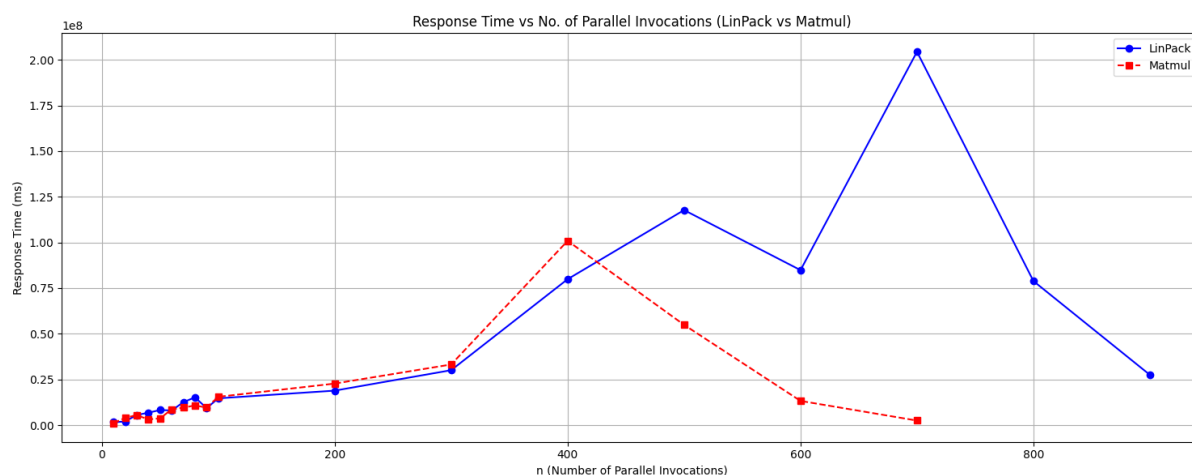


Figure 4.1: Comparative Analysis of the Workloads

4.3.1 Matrix Multiplication (Matmul)

As is evident in 4.1, a rough increase was observed in the response time as the number of parallel invocations were increased. However, this is saturated at ~ 400 invocations. This is because both the invokers did not have any more free memory for more containers and subsequent operations happen only when the minimum memory required by the `matmul` activation is freed in any of the invokers.

Even though the configurations were updated, we could not schedule more than 700 concurrent invocations for this activation without crashing the nodes and in some runs, the host.

We also tracked the CPU utilisations by the principal containers i.e. the scheduler, container, kafka and the two invokers. They are presented in 4.2 and 4.3. The figures presented are for the highest feasible number of concurrent invocations for this activation, 700.

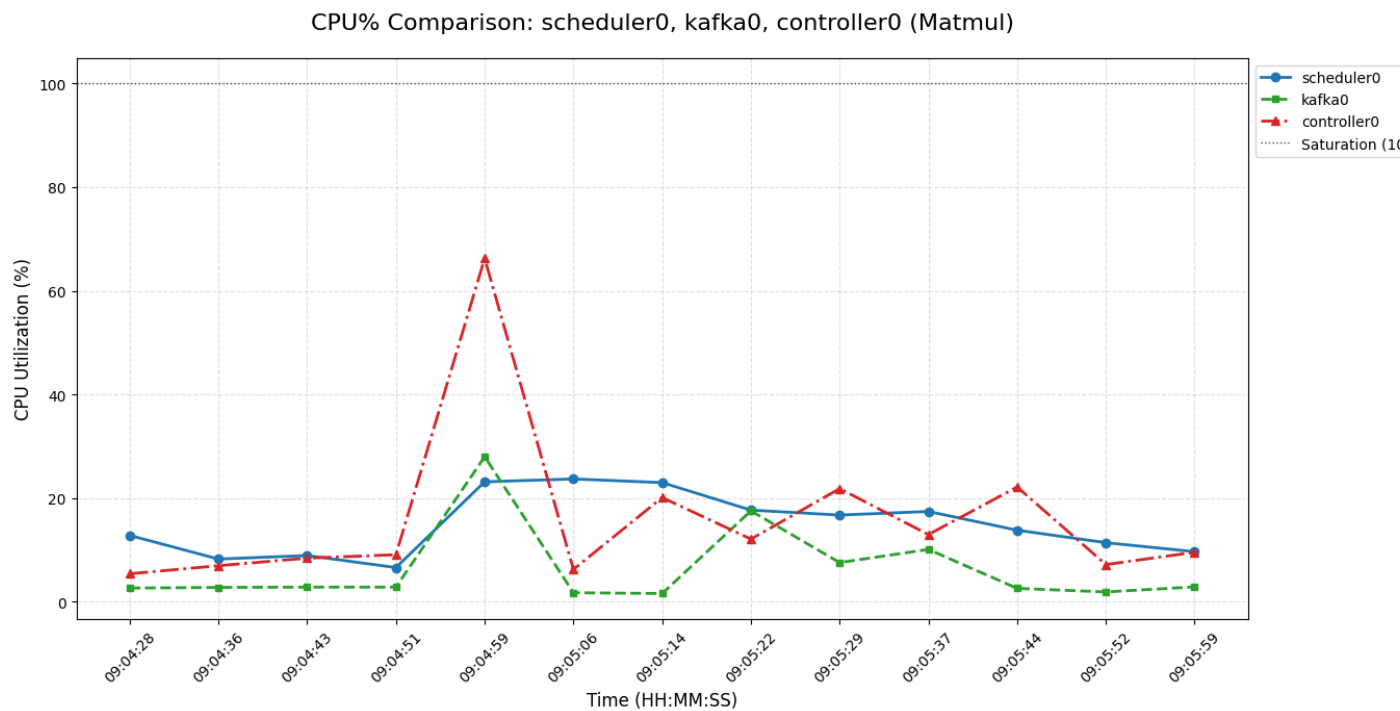


Figure 4.2: CPU Utilisation of Scheduler, Kafka and Controller containers

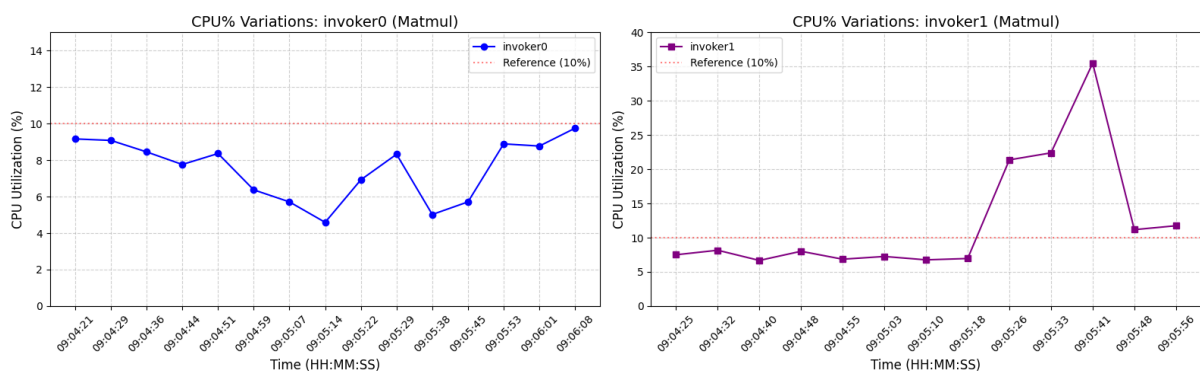


Figure 4.3: CPU Utilisation of Invokers

It can be concluded that the division of labour among the two invoker nodes is not equal even though they start at roughly the same utilisation. However, eventually, `invoker1` has been allocated more workload by the scheduler.

The initial spike in the controller utilisation can be attributed to the fact that both nodes were being assigned the activations and were also responding once they were completed. We speculate that the subsequent lows, are due to having only one invoker node bearing the majority of the load.

4.3.2 Solving Linear Equations (Linpack)

We were able to fire 900 concurrent `linpack` activations. After a rough increase in time, a saturation point is reached at ~ 200 invocations. The most evident outlier is 700. We speculate that this is due to scheduling of the activations of pre-warmed containers before they were removed. This may also be the reason for the subtle response time variations observed across runs.

We also tracked the CPU utilisation of the principal nodes for this workload for 900 invocations. They are shown in 4.6 and 4.5.

We can correlate the CPU usage spiked in these two figures quite clearly. It can be noted that both the ivoker nodes reach their peak from 08:35:00 to 08:36:00 hrs.

Subsequently, in a short duration utilstaions of the scheduler, kafka and controller nodes spike altogether due to significant response traffic sent by both invoker nodes.

After this, there is a significant drop in the CPU utilisation of all the principle nodes as the responses have been sent to the users and memory is now free. Although it must be noted that `invoker0` has incurred a higher CPU utilisation post spike.

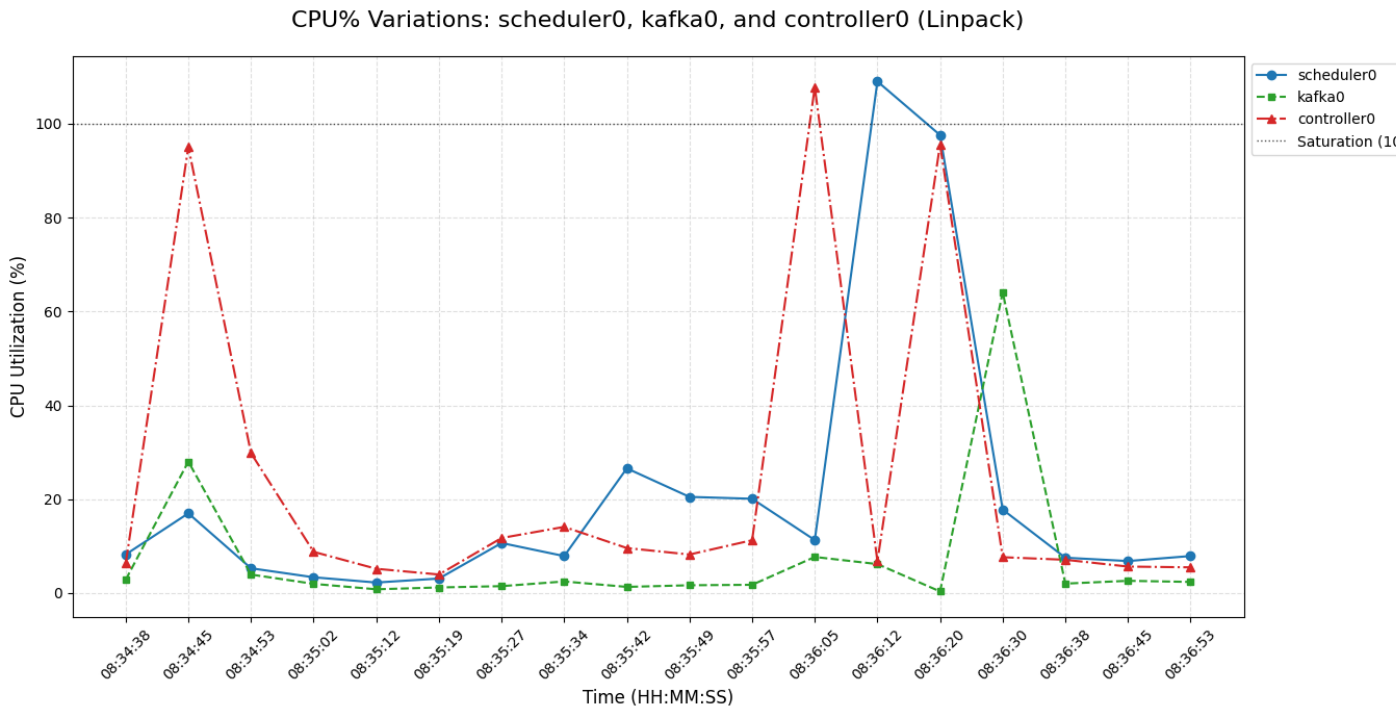


Figure 4.4: CPU Utilisation of Scheduler, Kafka and Controller containers

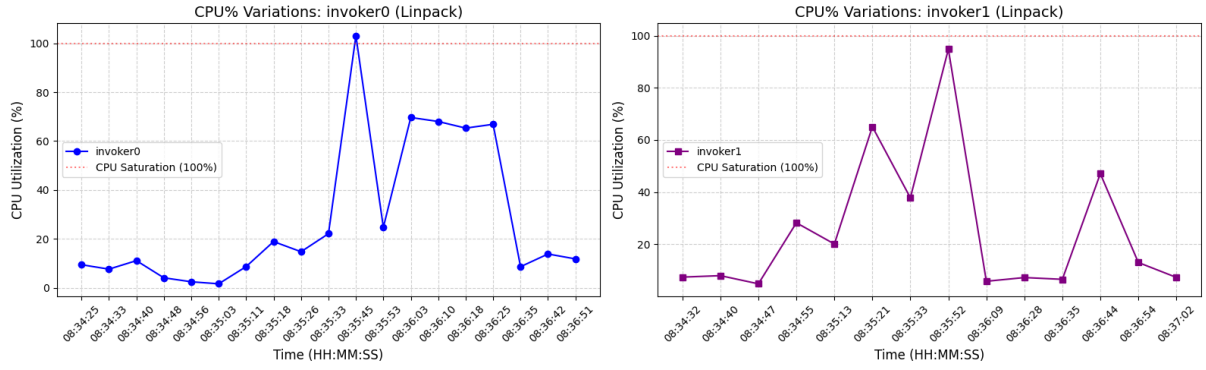


Figure 4.5: CPU Utilisation of Invokers

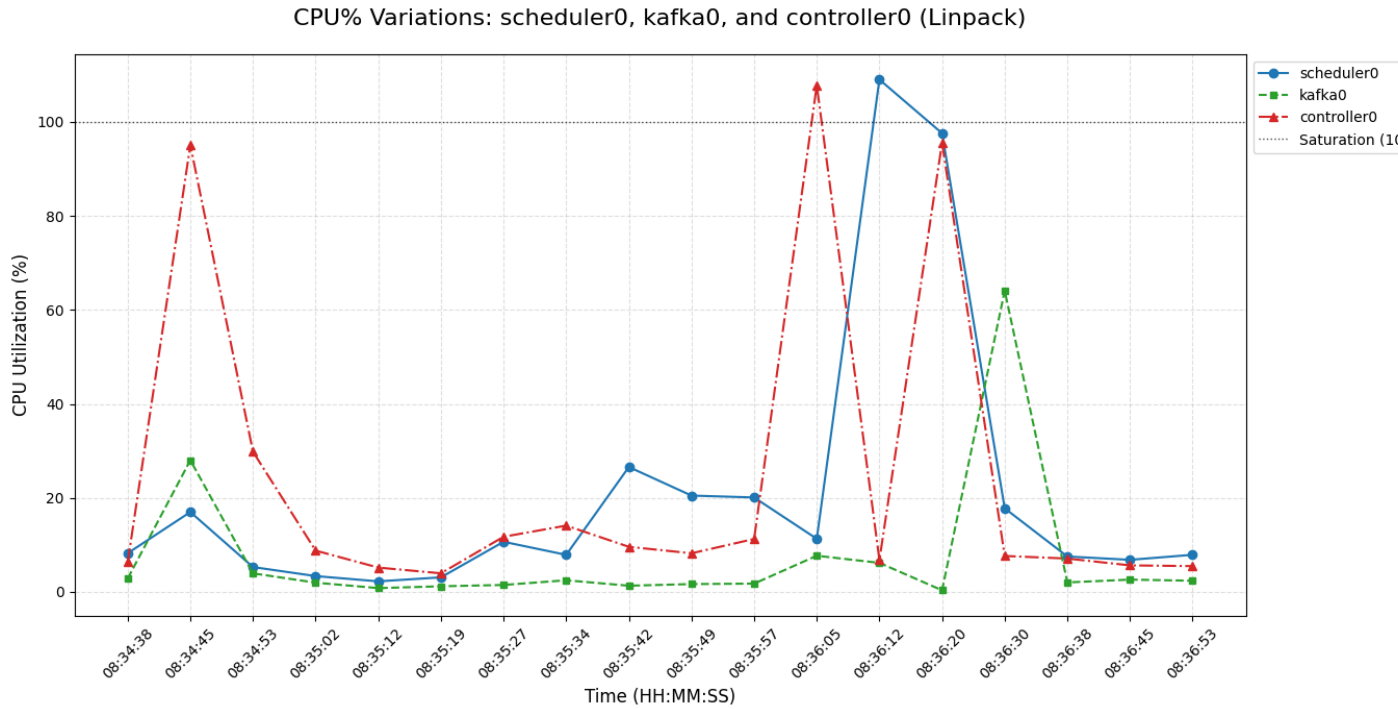


Figure 4.6: CPU Utilisation of Scheduler, Kafka and Controller containers

4.3.3 Inferences

We have made the following inferences based on our profiling of our OpenWhisk setup using two existing workloads:

- The division of labour among the invoker nodes is equitable. Initially a single invoker node handles all the invocations. Subsequently when that invoker node has reached its full capacity (i.e. no more containers can be created), the other invoker node handles the activations. Following this, the requests are then equally divided.
- Having single instances of the principal nodes (especially the controller), results in increased workload for all three containers. A more equitable distribution would be

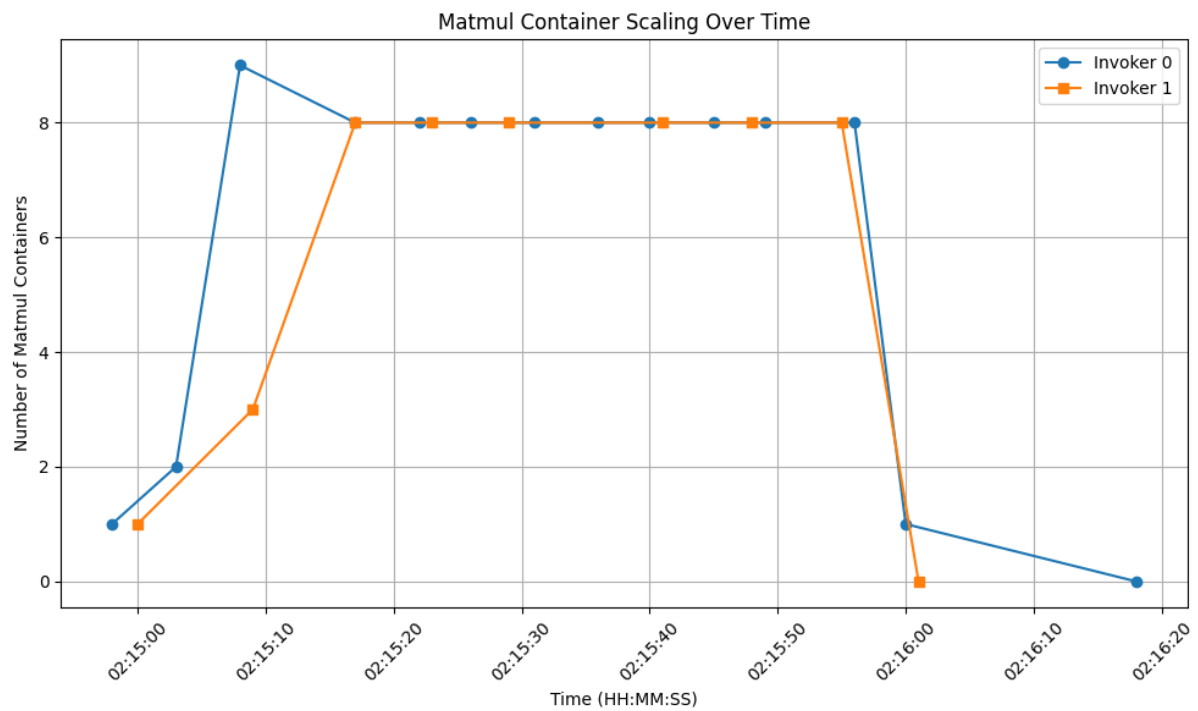


Figure 4.7: Number of containers created in invoker nodes for Matmul (over 500 parallel invocations)

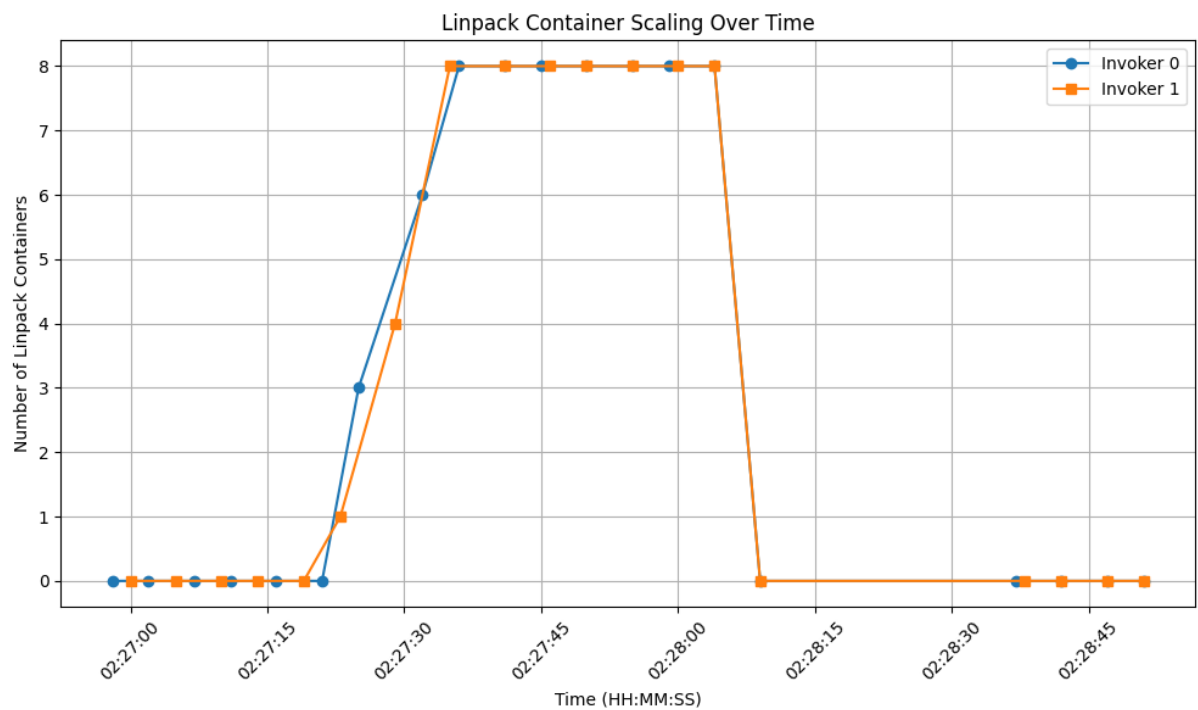


Figure 4.8: Number of containers created in invoker nodes for Linpack (over 500 parallel invocations)

to have more instances of controller nodes.

Chapter 5 Implementation

5.1 Creating Docker Container for C Support

OpenWhisk has the support for using custom Docker images at action runtime. Though the Docker image should specify certain constraints. To simplify our implementation, we used an existing Python image `action-python-v3.10:nightly` that already has support for C.

5.1.1 Approach 1

Using custom docker image provided by OpenWhisk

- Import ‘openwhisk/dockerskeleton’ image from Docker images repository. Dockerskeleton is a base image provided by OpenWhisk that serves as a minimal, lightweight environment.
- Inside Dockerfile, write command to install gcc and libc-dev (development libraries for C) into the image.
- Inside Dockerfile, write command to compile the C program inside image to produce its executable.
- Build the image and run it to get executable of C program.
- Add generated executable into a zip folder (say `action.zip`).
- Create openwhisk action on the zip folder

```
wsk action create c-binary action.zip --native
```

`--native` tells OpenWhisk that this action is a native executable rather than a script. OpenWhisk will then invoke this binary directly.

5.1.2 Approach 2

Configuring PATH in the environment of python container

- PATH environment variable is a colon-separated list of directories that the shell searches through to find executable programs when a command is entered.

```
1      env["PATH"] =  
2      "/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

- Give full path to gcc while compiling the C program as a subprocess.

```
1      compile = subprocess.run(  
2          ["/usr/bin/gcc", "hello.c", "-o", "hello"],  
3          capture_output=True, text=True, env=env  
4      )
```

5.1.3 Approach 3

A custom Docker image for Python was provided by [1]. We customised this image to specify a specific version of C for use in the Dockerfile. An image was then built, uploaded to Docker Hub and made public. This approach gives more flexibility over versioning.

5.2 Implementing the Activation Code(s)

We implement three types of activation codes as we explore three different cases to impose time limits. Firstly, the C code sent as a string is parsed and stored in .c file in a temporary directory. Secondly, the file is then compiled using gcc present in the container. These steps are common to all three approaches. In the third step, the code is run using the `subprocess.run()` method provided by Python.

5.3 Imposing Time Limits on Executions

We implement three variations of timeout. These are :

1. **Timing out the entire container.** This approach kills the entire container if it exceeds the specified time limit for execution.
2. **Timing out a specific test case.** In this approach, instead of killing the entire container only those executions that have exceeded the specified time limit will be terminated.

3. **Customising time limits for each test case.** In this approach, we can specify the time limit for each test case of an execution.

5.3.1 Timing out the entire Container

OpenWhisk supplies a parameter `timeout` when creating an activation. It is set to 60000 (milliseconds) by default. It can be customised when creating an activation :

```
wsk action create <action-name> <action-code> --timeout 100 -i
```

An error message is generated if the activation exceeds the time limit.

Container once timed out is not reused even if same action invoked just after.

5.3.2 Timing out a Specific Test Case

In this case if any test case exceeds the specified limit, a payload indicating the timeout is generated for that particular test case and the corresponding container then exits normally. It is especially useful if a code is stuck in an infinite loop for any test case. The desired time limit (in milliseconds) is passed as parameter in the payload along with the C code and the corresponding test cases :

```
1 {  
2     "code": "<c code>",  
3     "test-cases": [ ... ],  
4     "time-limit": 30 #(in ms)  
5 }
```

If, for any test case, the time limit exceeds the following response is returned :

```
1 {  
2     "compilation_successful": true,  
3     "results": [  
4         {  
5             "output": "...",  
6             "success": true,  
7             "test_case": 0  
8         },  
9         {  
10            "output": "...",  
11            "success": true,  
12            "test_case": 1  
13        },  
14        {  
15            "error": "Timed out", #Timed out test case
```



```
16         "success": false,
17         "test_case": 2
18     }
19 ],
20 "success": true
21 }
```

5.3.3 Customising Time Limits for each Test Case

In this case, we can specify a time limit for every execution test case. It can be useful in testing the scalability of code or for enduring an optimal code from the user's end. The following payload is required as input :

```
1 {
2     "code": "...",
3     "test-cases": [ ... ],
4     "time-limits": [
5         3,
6         3,
7         10,
8         ...
9     ]
10 }
```

If for any test case, the time limit exceeds, the response is same as described in the previous case.

Chapter 6 Results

6.1 Testing Timeout Scenarios

C files are taken from solutions to various coding challenges available at [LeetCode](#) and [GeeksForGeeks](#). We have also provided faulty codes and buggy test cases that cause the compilation to fail, cause the code to run into infinite loop and/or result in a runtime error.

The C files are saved at the client machine along with the `.csv` files for the test cases and time limits (if applicable). A Python script takes all the `.c` and `.csv` files to create an appropriate JSON payload to pass to the OpenWhisk activation.

We evaluated the different ways we impose the time limits for several test cases. A few of them are mentioned here as examples.

Successful Run

This is common to all three timing cases. For example, we take the C code to find if a string of parentheses is balanced or not (can be found [here](#)). The input payload is :

```
1 {
2     "code": "<c code as mentioned in the link>",
3     "test-cases": [
4         "()()()",
5         "()",
6         "()"
7     ]
8 }
```

As all the test cases are valid and the code is not buggy, we get the following response :

```
1 {
2     "compilation_successful": true,
3     "results": [
4         {
5             "output": "true",
6             "success": true,
```

```
7         "test_case": 0
8     },
9     {
10         "output": "true",
11         "success": true,
12         "test_case": 1
13     },
14     {
15         "output": "true",
16         "success": true,
17         "test_case": 2
18     }
19 ],
20 "success": true
21 }
```

Unsuccessful Run

We try to run a code that is guaranteed to cause a compilation failure. In this case, the code has a missing “;”. The input payload is :

```
1 {
2     "code": "#include<stdio.h>\nint main(){\nint
3     a\nscanf(\"%d\",&a);\nreturn 0;\n}\n",
4     "test-cases": [
5         "1",
6         "2",
7         "3",
8         "4",
9         "5"
10    ]
11 }
```

The output payload is :

```
1 {
2     "compilation_error": "/tmp/tmp4ubpcwvb/program_3112a7xxx.c:
3     In function
4     'main':\n/tmp/tmp4ubpcwvb/program_3112a7xxx.c:4:1: error:
5     expected '=', ',', ';', 'asm' or '__attribute__' before
6     'scanf'\n ...",
7     "success": false
8 }
```

Partially Successful Run

We provide three test cases here out of which one causes the code to run into infinite loop. The input payload is :

```
1 {
2   "code": "#include<stdio.h>\nint main(){\nint
3     a;\nscanf(\"%d\",&a);\nwhile(a>2){}\nprintf(\"Out of the
4     loop\");\n}\n",
5   "test-cases": [
6     "1 20",
7     "2 10",
8     "3 10"
9   ],
10  "time-limits": [
11    100,
12    200,
13    300
14  ]
15 }
```

Note that the third test case would cause an infinite loop. Hence, the output payload is :

```
1 {
2   "compilation_successful": true,
3   "results": [
4     {
5       "output": "Out of the loop",
6       "success": true,
7       "test_case": 0
8     },
9     {
10      "output": "Out of the loop",
11      "success": true,
12      "test_case": 1
13    },
14    {
15      "error": "Timed out after 300 ms",
16      "success": false,
17      "test_case": 2
18    }
19  ],
20  "success": true
21 }
```

21 }

6.2 Profiling the Implementation

We used 10 C files each with the same properties as mentioned in the previous section. This is done to emulate a real time scenario and create a more heterogeneous workload for profiling the setup. The timings are averaged over 5 runs for every number of parallel invocations.

6.2.1 Evaluating the Timing

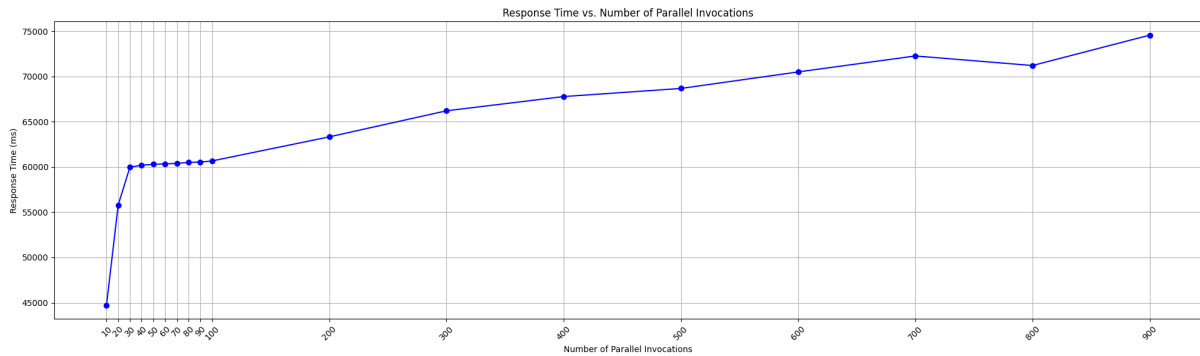


Figure 6.1: Response Time for various Parallel Invocations

6.2.2 Evaluating the CPU Usage

There are synchronized spikes and dips between the invoker activity and the system component load. This is evident in Figures 6.2 and 6.3.

CPU usage of `invoker1` surges sharply to $\sim 40\%$ initially and `invoker1` spike appears to precede or coincide with growing scheduler and controller load. This suggests that the heavy workload on `invoker1` triggered control-plane activity: the scheduler and Kafka pick up tasks or messages as the invoker starts work, and then the controller ramps up (processing results or managing state) right after the invoker's burst. Subsequently, `invoker0` spikes to $\sim 14\%$ while `invoker1` is moderate ($\sim 7\%$). Scheduler likewise peaks ($\sim 25\%$) and controller rises to $\sim 15\%$. These nearly simultaneous rises suggest a coordinated event: a workload that drove both invoker execution and scheduling efforts. The controller's usage is smaller, indicating it might be catching up after scheduler decisions. In this case the invoker (`invoker0`) peak and scheduler peak occur together, implying concurrent workload initiation and scheduling.

There is also a major concurrent spike at about 00:13:00, `invoker1` jumps to $\sim 16\%$ and

invoker0 to $\sim 8\%$. At this same moment, the scheduler surges to $\sim 53\%$ and the controller to almost $\sim 70\%$. All these components peak together in close alignment. This alignment strongly indicates that a new workload or function invocation caused all components to work simultaneously: the invokers are busy running tasks while scheduler and controller handle scheduling decisions and state updates. The roughly coincident peaks suggest tight coupling of invoker execution with control-plane processing. Following

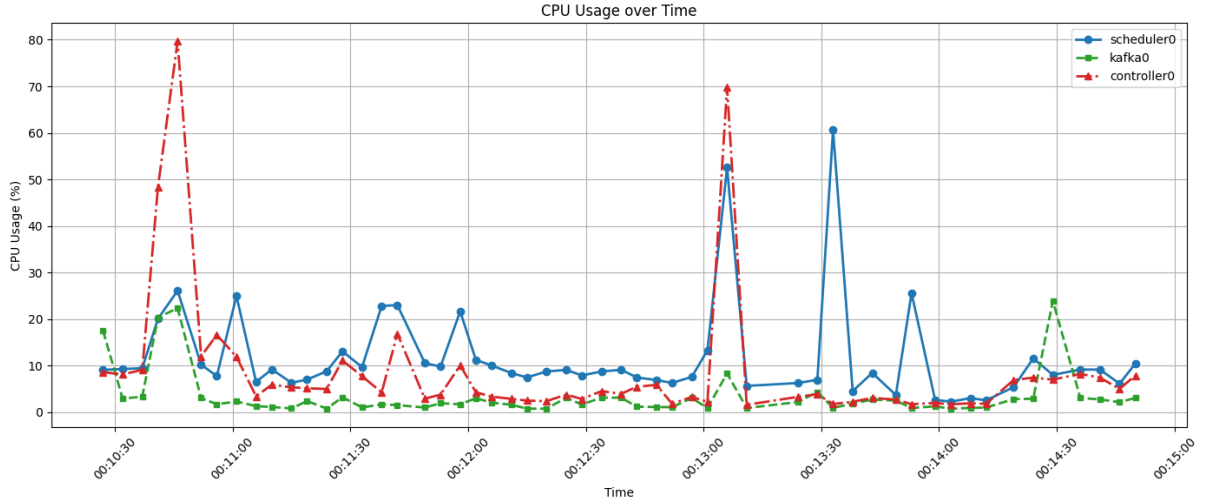


Figure 6.2: CPU Utilisation of Scheduler, Kafka and Controller containers

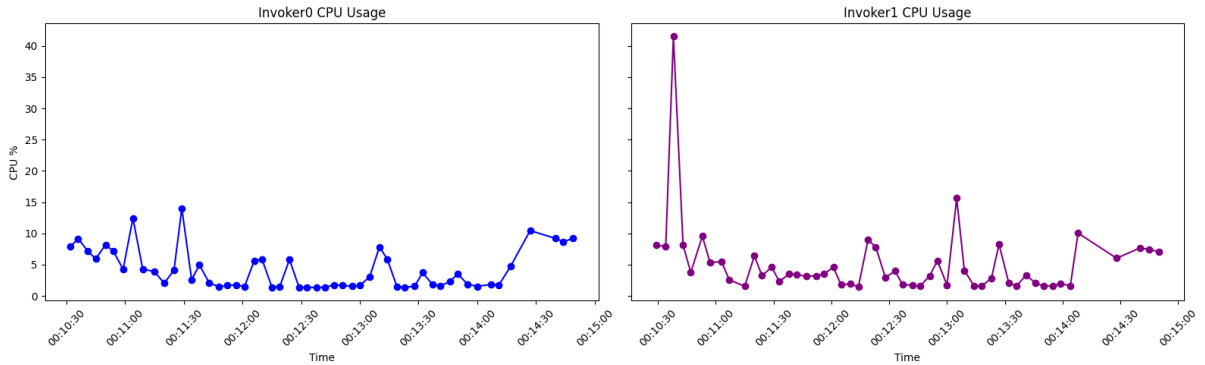


Figure 6.3: CPU Utilisation of Invokers

this, around 00:13:30 the scheduler jumps to $\sim 62\%$ CPU, even though the invokers are low (around 2–10%) and controller is nearly idle. This is an anomaly where the scheduler spikes without an obvious concurrent invoker burst. It could indicate delayed processing – for example, the scheduler catching up on pending tasks generated by earlier invocations – or an independent scheduling operation (such as rebalancing or cleanup). The lack of a matching invoker spike suggests this scheduler load is not directly caused by immediate invoker CPU usage.

In the final window around 00:14:30, both invokers show moderate activity and all three components have modest upticks. These smaller, concurrent peaks imply a lower-intensity

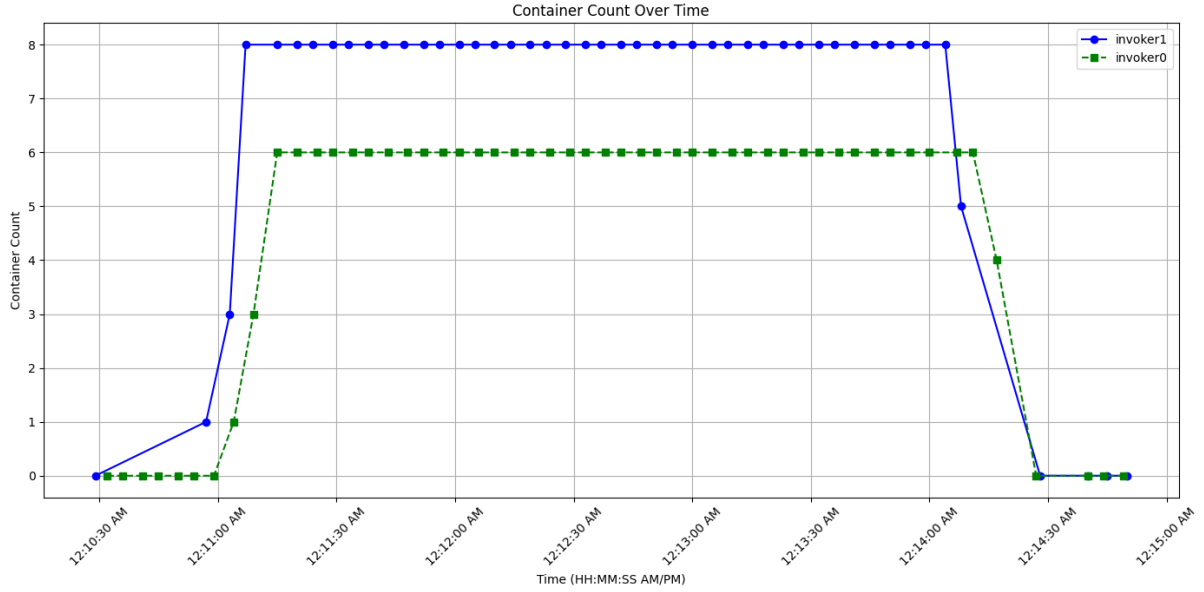


Figure 6.4: Number of containers created during 900 parallel invocations

workload occurring across the system. All components rise together moderately, indicating a synchronized but lighter task load affecting invokers and control-plane.

Across the timeline, Kafka node's CPU (green) remains very low except for two small bumps ($\sim 18\%$ at 00:10:50 and $\sim 8\%$ at 00:14:30). It does not spike as dramatically as the other components. Kafka's usage seems only loosely aligned with the invoker peaks: it jumps with the initial burst but stays near baseline otherwise. This suggests that Kafka's messaging role generates comparatively little CPU overhead in this workload, and its peaks may lag or lead invoker activity by a small margin. It can be concluded that Kafka does not cause a major bottleneck on even heavier workloads. It can also be concluded that, if possible, there should be a 1:1 mapping between number of containers and number of invokers.

As indicated in figure 6.4, even with a heterogeneous workload, the balance between the two invoker nodes is somewhat equitable. Since there are different kinds of C codes and hence have different memory usages which may have factored in the invoker node selection giving rise to small difference. This is unlike figures 4.7 and 4.8 where each node was doing the same amount of work throughout the run.

Chapter 7 Conclusions and Future Work

We have successfully installed OpenWhisk on multiple Ubuntu servers and also setup the client machine to use it. We have studied its workflow, load balancer policy and the interplay of the controller, scheduler and invoker nodes. We have also profiled the setup with two existing benchmarks and drawn inferences from it. Based on our understanding, we have designed a setup that supports running several C code executions on each of the invoker nodes and imposed time limits on those executions in different ways. We profiled our implementation on a heterogeneous workload of several different C codes.

For future work, we aim to scale our setup to multiple nodes and integrate it with Prutor, which is used for conducting ESC111/2 labs at IIT Kanpur. We also plan to add support for multiple languages, such as Java and Python. Furthermore, we seek to enable GPU container support within the OpenWhisk setup.

Bibliography

- [1] Christos Katsakioris et al. “FaaSRAil: Employing Real Workloads to Generate Representative Load for Serverless Research”. In: *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*. HPDC '24. Pisa, Italy: Association for Computing Machinery, 2024, pp. 214–226. ISBN: 9798400704130. DOI: [10.1145/3625549.3658684](https://doi.org/10.1145/3625549.3658684). URL: <https://doi.org/10.1145/3625549.3658684>.
- [2] Jeongchul Kim and Kyungyong Lee. “FunctionBench: A Suite of Workloads for Serverless Cloud Function Service”. In: *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. 2019, pp. 502–504. DOI: [10.1109/CLOUD.2019.00091](https://doi.org/10.1109/CLOUD.2019.00091).